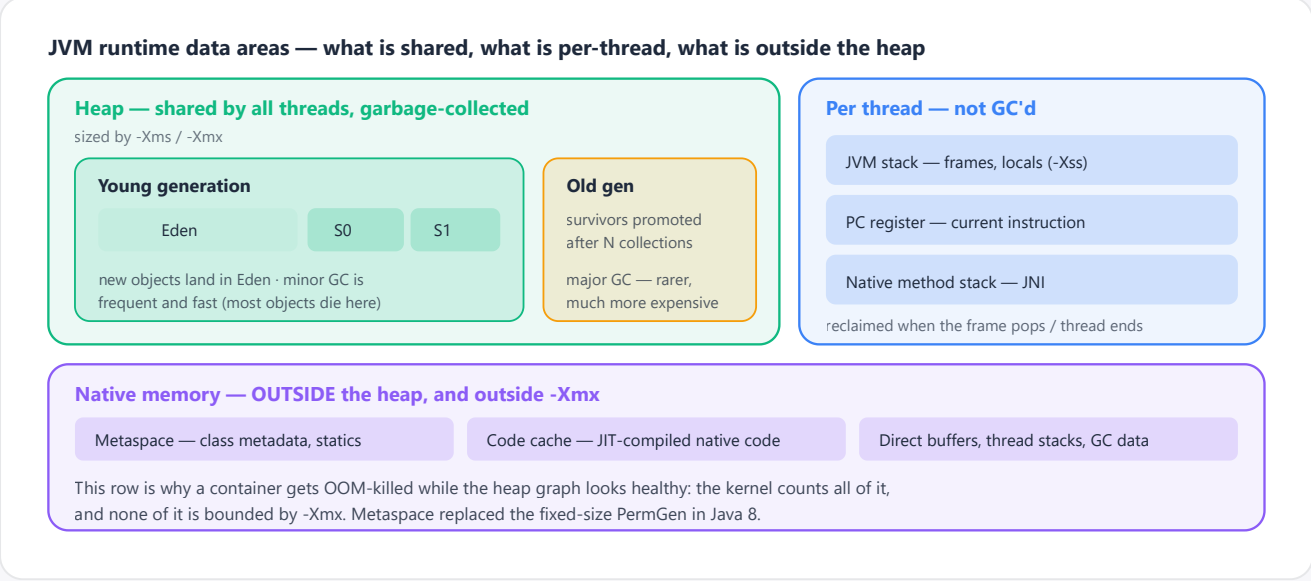
JVM Memory Areas

Why it matters

Chapter 1 introduced stack and heap as a programming model. This topic is the operational version: which regions exist, which are bounded by which flag, and why a container can be killed for using too much memory while the heap graph looks perfectly healthy. That last question is one of the most common real production puzzles in Java, and it has a precise answer.

The concept, simply

The JVM divides memory into regions with different lifetimes and owners:



The bottom band is the one that surprises people: Metaspace, the code cache, thread stacks and direct buffers all live outside the heap and are not bounded by -Xmx.

How it works

Heap	All threads	Every object and array	`-Xms` / `-Xmx`
JVM stack	Per thread	Frames, locals, operand stack	`-Xss` × thread count
PC register	Per thread	Current instruction address	Negligible
Metaspace	All threads	Class metadata, statics	`-XX:MaxMetaspaceSize` (unbc
Code cache	All threads	JIT-compiled native code	`-XX:ReservedCodeCacheSize`
Direct buffers	All threads	NIO off-heap buffers	`-XX:MaxDirectMemorySize`

The heap is generational — young (Eden plus two survivor spaces) and old — because most objects die young. Allocation in Eden is a pointer bump inside a thread-local buffer, which is why object creation is far cheaper than most people assume (Chapter 1).

Metaspace replaced PermGen in Java 8. PermGen was a fixed-size region inside the heap; Metaspace is native memory that grows on demand. That removed the old `OutOfMemoryError: PermGen space` but introduced a new failure mode — unbounded growth from class-loader leaks that can consume all system memory unless you cap it.

Code examples

Sizing a JVM in a container, which is where this knowledge pays:

BASH Copy

```
# ✗ Naive: container limit 1 GB, so give the heap 1 GB. The process is
#   OOM-killed within hours — stacks, Metaspace, code cache and GC
#   structures all live OUTSIDE -Xmx and the kernel counts every byte.
java -Xmx1g -jar app.jar

# ✓ Leave headroom for the non-heap regions.
java -Xms700m -Xmx700m      -XX:MaxMetaspaceSize=256m      -XX:ReservedCod
```

```
# ... JVM read the container limit itself (Java 10+, and
# ... the default since 11). It sizes the heap as a PERCENTAGE of the limit.
java -XX:MaxRAMPercentage=70 -jar app.jar
# Older JVMs saw the HOST's memory, not the container's, and sized the hea
# far too large — the classic "works on my machine, OOM-killed in Kubernet
```

Seeing where memory actually goes:

BASH Copy

```
# Native Memory Tracking — the only way to see non-heap usage properly.
java -XX:NativeMemoryTracking=summary -jar app.jar
jcmd <pid> VM.native_memory summary

# Output names each region — Java Heap, Class (Metaspace), Thread, Code,
# GC, Compiler, Internal — so you can see which one is actually growing.

jcmd <pid> GC.heap_info        # generation sizes and occupancy
jcmd <pid> VM.flags            # every flag in effect, including defaults
jstat -gcutil <pid> 1000       # live GC statistics, once per second
```

The four `OutOfMemoryError` messages, which have different causes:

TEXT Copy

```
java.lang.OutOfMemoryError: Java heap space
  → too many live objects. A leak, or an undersized heap. Take a heap dump.

java.lang.OutOfMemoryError: Metaspace
  → too many classes. Usually a class-loader leak from repeated redeploys,
    or a framework generating proxies without bound.

java.lang.OutOfMemoryError: unable to create new native thread
  → the OS refused a thread. Thread leak, or the per-process limit.
    NOT a heap problem — raising -Xmx makes it WORSE (less room for stacks).

java.lang.OutOfMemoryError: GC overhead limit exceeded
```

Under the hood

TLABs — Thread-Local Allocation Buffers — are why allocation needs no synchronisation. Each thread gets a private slice of Eden and bumps a pointer within it; only when the slice is exhausted does it take a slower shared path. This is the mechanism behind "allocation is a handful of instructions".

Compressed oops is the detail with the counter-intuitive consequence. On 64-bit JVMs with a heap under about 32 GB, references are stored as 32-bit offsets rather than 64-bit addresses, halving reference size and improving cache behaviour. Cross that threshold and compressed oops switch off, every reference doubles, and **raising `-Xmx` from 31 GB to 33 GB can reduce the number of objects you can hold**. The practical advice is to stay under 32 GB, or jump well past it.

The heap does not shrink readily. Traditionally the JVM returned memory to the OS reluctantly, which is why `-Xms` equal to `-Xmx` is common on servers — it avoids resize pauses and makes usage predictable. G1 and ZGC have improved at giving memory back, but on a container with a hard limit, fixed sizing remains the safer choice.

Common mistakes

- **Setting `-Xmx` to the container limit.** Ignores every non-heap region; the process gets OOM-killed.
- **Assuming `-Xmx` bounds total memory.** Metaspace, stacks, code cache and direct buffers are all outside it.
- **Raising `-Xmx` for "unable to create new native thread".** That takes memory away from stacks and makes it worse.
- **Ignoring which OOM message you got.** Four messages, four different causes and fixes.
- **Leaving Metaspace unbounded.** A loader leak can consume all system memory.

- **Crossing the 32 GB compressed-oops threshold accidentally.** Capacity can go down.
- **Running an old JVM in a container.** Before Java 10 it saw the host's memory and sized the heap far too large.

Interview lens

Q: "What are the JVM memory areas?"
A: Shared across threads: the heap, which holds every object and is generational; Metaspace for class metadata and statics; the code cache for JIT output. Per thread: the JVM stack of frames, the PC register, and a native method stack. The important operational point is that only the heap is bounded by `-Xmx` — everything else is native memory outside it.

Q: "Your container has a 1 GB limit. What do you set `-Xmx` to?"
A: Not 1 GB. Thread stacks, Metaspace, the code cache, GC structures and direct buffers all live outside the heap, and the kernel counts them, so the process would be OOM-killed. I would leave 25–30% headroom — around 700 MB of heap — and cap Metaspace and the code cache explicitly. On Java 10+ the cleaner approach is `-XX:MaxRAMPercentage`, letting the JVM read the container limit itself.

Q: "What replaced PermGen and why?"
A: Metaspace, in Java 8. PermGen was a fixed-size region inside the heap that was hard to size — too small and you got `OutOfMemoryError: PermGen space` on class-heavy applications, too large and you wasted heap. Metaspace lives in native memory and grows on demand, which removed that failure at the cost of a new one: unbounded growth from class-loader leaks, so it is worth capping.

Q: "You get `unable to create new native thread`. What is wrong?"
A: The OS refused to create a thread — either a thread leak in the application or a per-process limit. It is not a heap problem, and raising `-Xmx` actively makes it

reduce `-Xss` or move to virtual threads.

Q: "Why is a 31 GB heap sometimes better than a 33 GB one?"
A: Compressed oops. Under roughly 32 GB the JVM stores references as 32-bit offsets; above it they become full 64-bit pointers. Every reference doubles in size, so the extra 2 GB of heap can hold fewer objects than the smaller heap did. Stay comfortably under 32 GB, or go well beyond it.

Q: "How do you see non-heap memory usage?"
A: Native Memory Tracking — start with `-XX:NativeMemoryTracking=summary` and query it with `jcmd <pid> VM.native_memory summary`. It breaks usage down by category — heap, class, thread, code, GC, compiler — which is the only reliable way to answer "the heap is fine, so where is the memory going?".

Cheat-card

- Shared: **heap** (`-Xmx`), **Metaspace**, **code cache**. Per thread: **stack** (`-Xss`), PC register.
- `-Xmx` **bounds the heap only.** Container limits must allow ~25–30% headroom.
- Java 10+: `-XX:MaxRAMPercentage` — the JVM reads the container limit.
- Heap = young (Eden + S0/S1) + old. Allocation = **pointer bump in a TLAB**.
- **Metaspace replaced PermGen** in Java 8 — native, grows on demand, worth capping.
- Four OOMs: *heap space* · *Metaspace* · *unable to create native thread* · *GC overhead limit*. Different causes.
- **Compressed oops cliff at ~32 GB** — bigger heap, fewer objects.
- Diagnose non-heap with **Native Memory Tracking** + `jcmd VM.native_memory`.
- Servers: set `-Xms` = `-Xmx` to avoid resize pauses.

1. Your container has a 1 GB memory limit. Why is -Xmx1g wrong?

- ◯ Because the JVM cannot address a full gigabyte
- ◯ Because -Xmx bounds only the HEAP — thread stacks, Metaspace, the code cache and direct buffers are native memory outside it, and the kernel counts every byte
- ◯ Because -Xmx must always be a power of two
- ◯ Because the container limit includes the operating system

Check answer

2. You get 'OutOfMemoryError: unable to create new native thread'. What should you NOT do?

- ◯ Check the per-process limit with ulimit
- ◯ Look for a thread leak in the application
- ◯ Raise -Xmx — that takes memory away from thread stacks and makes it worse
- ◯ Take a thread dump to count threads

Check answer

3. Why can raising the heap from 31 GB to 33 GB let you hold FEWER objects?

- ◯ Because Metaspace shrinks proportionally
- ◯ Because the JIT disables inlining on large heaps
- ◯ Because the garbage collector runs less often

Compressed oops switch off above roughly 32 GB, so every reference doubles from 32 to 64 bits

Check answer

Previous
Class Loading and the Delegation Model

Next
Garbage Collection

LEARN MORE

ARTICLES & DOCS

- Run-time data areas (JVMS §2.5)
  docs.oracle.com
- java launcher options — man page
  docs.oracle.com

VIDEOS



Java 8 JVM Architecture Explained 🔥 | Heap, Stack, Metaspace (PermGen?) & JVM Memory | Part 2
Vaibhav Code Lab



Heap Space vs. Stack Memory in Java | Call Stack Explained | Memory Allocation | Geekific
Geekific

Hand-picked references for this lesson.